

Podstawowe procedury sieciowe w systemie Linux oraz Cisco iOS

O książce

Zwięzły przewodnik po procedurach używanych podczas laboratoriów z sieci komputerowych. Zawiera praktyczne polecenia, schematy postępowania i krótkie przypomnienia dotyczące adresacji IPv4, routingu, VLAN-ów, iptables, NAT oraz podstawowej diagnostyki w środowiskach Linux i Cisco IOS.

Informacje wydawnicze

Autor: Jan Filipowski

Wydanie: 1

Miejsce i rok: Poznań, 2026

Seria: Co Złego To Nie My, tom 47

Exemplarz: do szybkiego ratowania konfiguracji

Trochę o systemie Linux

Linux jest systemem, który często wygląda jak test cierpliwości przebrany za wolność wyboru. Potrafi stać kilka dni bez dotykania, dać użytkownikowi złudzenie stabilności, a potem obudzić się z gorszym porankiem i obrazić na własny bootloader. GRUB wygląda wtedy jak czarny ekran z piwnicy informatyka z 1998 roku: `grub rescue>`, zero ludzkiego komunikatu, za to pełne zaproszenie do zgadywania, gdzie jest partycja, kernel i initrd.

Programy też mają osobny kabaret. Jedno jest jako `.deb`, drugie jako `.rpm`, trzecie jako Flatpak, czwarte jako Snap, piąte jako Appliance, szóste tylko z GitHuba, a siódme mówi „just compile from source”, jakby kompilacja zależności o trzeciej w nocy była normalną czynnością domową. Recovery mode nie jest ratunkiem, tylko ustnym egzaminem z administracji systemem: montowanie partycji, odbudowa `initramfs`, reinstalacja GRUB-a, podpisywanie modułów i symboliczna modlitwa do Tuxa. Linux nie jest wolnością. Jest escape roomem, który ktoś zainstalował zamiast systemu.

O programie GNS3

GNS3 to nie program, tylko cywilizowany dowód na to, że informatyka czasem potrafi nie nienawidzić użytkownika. Instalacja jest tak beczelnie prosta, że człowiek aż sprawdza, czy na pewno niczego nie pominął: pobierasz, uruchamiasz, konfigurujesz i nagle masz przed sobą środowisko do budowania laboratoriów sieciowych, a nie rytuał przywoływania zależności z otchłani systemu.

Interfejs GNS3 to czysta elegancja praktyczności. Masz workspace, urządzenia, linki i topologię, którą normalnie widzisz, rozumiesz i składasz jak człowiek. Marketplace i gotowe appliance'y zdejmują z użytkownika masę ręcznego dźbania, GNS3 VM bierze na siebie ciężką robotę emulacji, a całość daje poczucie, że budujesz profesjonalny poligon sieciowy bez szafy rackowej, płataniny kabli i wentylatorów wyjących jak odrzutowiec.

Jan Filipowski



Co Złego
To Nie My 47

Wn
Wydawnictwa
Naukowo-Techniczne
2024

Sieci komputerowe - zaliczenie laboratorium

Jak zacząć zadanie

1. **Narysuj topologię i podpisz porty.** Każdy kabel oznacz nazwami interfejsów, np. PC1 p4p1 -- p4p2 PC2. Od razu zaznacz, które urządzenie ma routować.
2. **Podziel sieć na domeny L2.** Jeden VLAN albo jeden przewód punkt-punkt to zwykle osobna podsieć IP. Nie mieszaj dwóch różnych segmentów w tej samej adresacji.
3. **Wpisz adresy na rysunku.** Przy każdym interfejsie zapisz IP/maskę; przy hostach zapisz bramę domyślną. Dla małych zadań /24 jest najczytelniejsze.
4. **Uruchamiaj od najbliższego sąsiada.** Najpierw link, potem IP, potem ping do urządzenia bezpośrednio połączonego. Dopiero potem trasy, forwarding, VLANy, NAT i firewall.
5. **Zmieniaj jedną rzecz naraz.** Po każdej zmianie sprawdź wynik: `ip addr`, `ip route`, `ping`, `sudo iptables -L -n -v`. Jeśli coś przestaje działać, wiesz po czym.
6. **Na końcu przygotuj dowód.** Prowadzącemu pokazujesz działający ping/ssh/ncat, trasę pakietu, liczniki iptables albo przechwycenie w Wiresharku.

Sprzęt w laboratorium

PC i patch panel

- em1 - główny interfejs, zwykle Internet/DHCP, w mostku br0; na patch panelu zwykle 4. gniazdo.
- p4p1 - port od okna/prawy; zwykle 1. gniazdo.
- p4p2 - port od drzwi/lewy; zwykle 2. gniazdo, czasem niepodpięty.
- /dev/ttyS0 - port szeregowy do konsoli Cisco; zwykle 3. gniazdo, płaski niebieski kabel.
- Kable: prosty do różnych warstw, np. PC-switch; krosowany do tej samej warstwy, np. switch-switch lub PC-router. Auto MDI-X często ratuje PC, ale na zapleczu lepiej użyć właściwego kabla.

```
# identyfikacja karty - miga dioda
sudo ethtool -p p4p1
ethtool p4p1
sudo ethtool -S p4p1      # statystyki karty
sudo ethtool -i p4p1      # sterownik/firmware
sudo ethtool -s p4p1 speed 10 duplex half autoneg off # tylko gdy zadanie żkae
ip link show
```

Cisco przez konsolę

```
picocom /dev/ttyS0      # zwykle 9600, stare/czarne
picocom -b 115200 /dev/ttyS0 # nowe/białe switche
# wyjście: Ctrl-a, potem q
```

Jeśli po Enter nic nie ma: zły patch panel, zły kabel, zły baudrate, urządzenie nie włączone. Reset: router zwykle przełącznikiem zasilania, switch przez odłączenie zasilania.

Checklista „nie działa”

1. **Fizycznie:** świeci port? Dobry patch panel? Dobry kabel? `ip link`, `sudo ethtool -p p4p1`, `Cisco show ip interface brief`.
2. **Adresy:** każda domena L2/VLAN ma własną sieć? Maski po obu stronach są zgodne? Nie został stary adres po poprzednim zadaniu?
3. **Sąsiad:** ping do bezpośredniego sąsiada działa? Jeśli nie, routing dalszy nie ma znaczenia.
4. **ARP:** `ip neigh` pokazuje MAC sąsiada/bramy? Dla zdalnego hosta ARP ma być do bramy, nie do hosta docelowego.
5. **Forwarding:** na Linux-routerze `sysctl net.ipv4.ip_forward` ma pokazać 1. Na Cisco routing między interfejsami L3 działa domyślnie.
6. **Trasy w obie strony:** źródło ma trasę do celu, każdy router zna następny hop, a cel wie, gdzie odesłać odpowiedź.
7. **Firewall/NAT:** sprawdź polityki, kolejność reguł i liczniki. Przy DNAT pamiętaj, że pakiet idzie przez FORWARD, nie INPUT.
8. **Usługa:** `ss -tunap` pokazuje proces na właściwym IP i porcie? Jeśli testujesz z sieci, samo 127.0.0.1 nie wystarczy.
9. **Wireshark:** jeśli widzisz pakiet tylko w jedną stronę, szukaj routingu powrotnego, filtra albo złego NAT.

Porty, SSH, ncat/netcat

```
ss -tunap
sudo ss -tunap | grep ':22'
sudo netstat -tunap | grep ':74'
systemctl status sshd
sudo systemctl start sshd
ssh -v user@192.168.1.10
ssh -vv user@192.168.1.10

# TCP: serwer i klient
ncat -l 192.168.1.55 1234
ncat 192.168.1.55 1234

# UDP
ncat -ul 192.168.1.55 1234
ncat -u 192.168.1.55 1234

# plik
ncat -l 192.168.1.55 1234 > out.txt
ncat 192.168.1.55 1234 < in.txt

# tryby pomocnicze z ncat
ncat -l 192.168.1.55 1234 --chat
ncat 192.168.1.55 1234 --chat
ncat -lk 192.168.1.55 8081 --max-conns 3 --allow 192.168.1.0/24
ncat -lk 192.168.1.55 8081 -c 'while true; do read i && echo [echo] $i; done'
ncat -o out.log -lk 192.168.1.55 8080 -c 'printf "HTTP/1.0 200 OK\r\n\r\n"; cat index.html'

# pseudo HTTP: serwowany plik HTML
printf '<html><body><h1>SK</h1></body></html>\n' > index.html
{ printf 'HTTP/1.0 200 OK\r\nContent-Type: text/html\r\n\r\n'; cat index.html; } \
| ncat -l 192.168.1.55 8080

# klient: reczne zapytanie GET
ncat 192.168.1.55 8080
GET / HTTP/1.0

# klient HTTP do zewnetrznego serwera
ncat www.cs.put.poznan.pl 80
GET /tkobus HTTP/1.0

# test przepustowosci, jesli prowadzacy o to pyta
netserver -DN
netperf -H 192.168.1.55
```

Po wpisaniu GET / HTTP/1.0 naciśnij Enter dwa razy, bo pusta linia kończy nagłówki HTTP. Do testów DNAT usługa musi słuchać na adresie osiągalnym z sieci, np.192.168.111.1, a nie tylko na 127.0.0.1. Porty: 1–1023 są uprzywilejowane, 1024–49151 zarejestrowane, 49152–65535 efemeryczne.

Wireshark: co pokazać

- **Routing:** pakiet wychodzi jednym interfejsem routera i wraca drugim; traceroute pokazuje router po drodze.
- **VLAN:** hosty w tym samym VLANie pingują się; hosty w różnych VLANach bez routera nie. Na porcie access nie widać broadcastów z innych VLANów.
- **SNAT/MASQUERADE:** po stronie zewnętrznej źródłem jest adres routera, nie hosta prywatnego.
- **DNAT:** klient łączy się z adresem/portem routera, a pakiet po DNAT ma cel ustawiony na hosta za NATem.
- **DROP vs REJECT:** przy DROP klient czeka; przy REJECT szybciej widać odpowiedź ICMP albo TCP RST.

Broadcast do obserwacji VLAN/ARP: sudo arping -I p4p1 <nieuzyty-adres-w-mojej-sieci> albo ping do nieuzytego adresu w swojej podsieci.

Adresacja IPv4 i podsieci

Podstawy

Adres IPv4 = 32 bity. Maska /n mówi, ile bitów to sieć. Hosty w zwykłej sieci: $2^{32-n} - 2$ (bez adresu sieci i broadcastu). Adres sieci = IP AND maska. Broadcast = część hosta wypełniona jedynekami.

Pref.	Maska	Adresy	Hosty
/0	0.0.0.0	4 294 967 296	4 294 967 294
/1	128.0.0.0	2 147 483 648	2 147 483 646
/2	192.0.0.0	1 073 741 824	1 073 741 822
/3	224.0.0.0	536 870 912	536 870 910
/4	240.0.0.0	268 435 456	268 435 454
/5	248.0.0.0	134 217 728	134 217 726
/6	252.0.0.0	67 108 864	67 108 862
/7	254.0.0.0	33 554 432	33 554 430
/8	255.0.0.0	16 777 216	16 777 214
/9	255.128.0.0	8 388 608	8 388 606
/10	255.192.0.0	4 194 304	4 194 302
/11	255.224.0.0	2 097 152	2 097 150
/12	255.240.0.0	1 048 576	1 048 574
/13	255.248.0.0	524 288	524 286
/14	255.252.0.0	262 144	262 142
/15	255.254.0.0	131 072	131 070
/16	255.255.0.0	65 536	65 534
/17	255.255.128.0	32 768	32 766
/18	255.255.192.0	16 384	16 382
/19	255.255.224.0	8 192	8 190
/20	255.255.240.0	4 096	4 094
/21	255.255.248.0	2 048	2 046
/22	255.255.252.0	1 024	1 022
/23	255.255.254.0	512	510
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2
/31	255.255.255.254	2	0*
/32	255.255.255.255	1	1*

Szybkie liczenie podsieci

- Dla /n rozmiar bloku w ostatnim zmiennym oktecie to 256 –wartość maski w tym oktecie.
- Przykład /28: maska 240, blok 16: sieci kończą się na .0, .16, .32, .48, ... Hosty w 192.168.1.32/28: .33–.46, broadcast .47.
- Podział na p równych podsieci: pożycz $x = \log_2(p)$ bitów, nowa maska = stara + x .
- VLSM: sortuj wymagania od największej liczby hostów, przydzielaj kolejne najmniejsze pasujące bloki.
- Adresy prywatne: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.

Linux: interfejsy, IP, routing

Czyszczenie i konfiguracja IP

```
ip addr show
ip link show
sudo ip link set p4p1 up
sudo ip link set p4p1 down
sudo ip addr flush dev p4p1 # usuwa wszystkie adresy IP z interfejsu
sudo ip addr add 192.168.10.2/24 dev p4p1
sudo ip addr add 10.0.0.2/24 dev p4p1 label p4p1:1 # dodatkowy adres
sudo ip addr del 192.168.10.2/24 dev p4p1
sudo ip link set p4p1 address aa:bb:cc:dd:ee:ff
ethtool -P p4p1 # permanent MAC
```

```
# stare net-tools: czytaj/awaryjnie, preferuj ip
ifconfig p4p1
sudo ifconfig p4p1 192.168.10.2 netmask 255.255.255.0
sudo ifconfig p4p1:1 10.0.0.2 netmask 255.0.0.0
sudo ifconfig p4p1 0.0.0.0
```

Czytaj ip addr: nazwy interfejsów, MAC, UP/DOWN, adresy i sieci. Jeśli jest DOWN, podnieś. Jeśli brak LOWER_UP/lampki, sprawdź kabel/port.

Routing w Linuxie

```
ip route show
ip route get 8.8.8.8
sudo ip route add 172.16.0.0/16 via 192.168.1.1
sudo ip route del 172.16.0.0/16
sudo ip route add default via 192.168.1.1
sudo ip route replace default via 192.168.1.1
sudo ip route flush cache # ścyczy cache decyzji routingu; rzadko potrzebne
```

```
# stare route: tylko gdy musisz
route -n
sudo route add -net 172.16.0.0 netmask 255.255.0.0 gw 192.168.1.1
sudo route del -net 172.16.0.0 netmask 255.255.0.0
sudo route add default gw 192.168.1.1
```

Czytaj ip route:

- wpis bez via: sieć bezpośrednio podłączona; src to adres używany jako źródłowy;
- wpis z via: pakiet idzie do bramy; brama musi leżeć w sieci bezpośrednio podłączonej;
- default: trasa ostatniej szansy;
- wybór trasy: najdłuższa maska > niższy metric > pierwszy pasujący wpis.

Forwarding - Linux jako router

```
sudo sysctl -w net.ipv4.ip_forward=1
cat /proc/sys/net/ipv4/ip_forward
sysctl net.ipv4.ip_forward

# jeśli powyższe nie działa na danej maszynie:
sudo sysctl -w net.ipv4.conf.all.forwarding=1
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

Bez forwardingu Linux przyjmie pakiety do siebie, ale nie przekaże ich dalej. NAT nie zastępuje forwardingu.

Przekierowanie SSH na inny port

```
# PC2: port 1234 na PC2 -> SSH PC3:22
sudo iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 1234 \
-j DNAT --to-destination 192.168.111.1:22
# PC1:
ssh -p 1234 user@192.168.1.2
```

Jeżeli FORWARD DROP, dodaj:

```
sudo iptables -A FORWARD -i p4p1 -o p4p2 -p tcp -d 192.168.111.1 --dport 22 \
-m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A FORWARD -i p4p2 -o p4p1 -p tcp -s 192.168.111.1 --sport 22 \
-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

DNAT bez trasy zwrotnej? Dodaj SNAT

Jeśli host docelowy nie ma defaultu przez router NAT, odpowiedź pójdzie inną drogą albo zginie. Wtedy możesz wymusić, żeby dla DNAT-owanych połączeń źródłem był adres routera od strony hosta prywatnego:

```
# DNAT z PC1 do PC3
sudo iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 74 \
-j DNAT --to-destination 192.168.111.1:74
# SNAT odpowiedniego ruchu, żeby PC3 odpowiadał do PC2
sudo iptables -t nat -A POSTROUTING -o p4p2 -p tcp -d 192.168.111.1 --dport 74 \
-j SNAT --to-source 192.168.111.2
```

Mangle i TTL

Ustawienie TTL dla ruchu przechodzącego przez router

Przykład z zadania NAT: pakiety z PC3 192.168.111.0/24 do PC1 192.168.1.0/24 mają wychodzić z PC2 z TTL=1.

```
sudo iptables -t mangle -A POSTROUTING \
-s 192.168.111.0/24 -d 192.168.1.0/24 \
-j TTL --ttl-set 1
sudo iptables -t mangle -L -n -v -x --line-numbers
```

Uwaga: W POSTROUTING ustawiasz TTL tuż przed wyjściem. Host docelowy może zobaczyć TTL=1; gdyby po drodze był kolejny router, pakiet by wygaśł. Do obserwacji użyj Wiresharka.

Testowanie i debugowanie

Ping, trasa, liczniki

```
ping -c 3 <IP_SASIADA>
ping -c 3 <IP_KONCOWEGO_HOSTA>
ping -b <BROADCAST_SIECI>
ping -s 4000 <IP_KONCOWEGO_HOSTA>
traceroute -I <IP_KONCOWEGO_HOSTA> # ICMP
traceroute -T <IP_KONCOWEGO_HOSTA> # TCP
traceroute -U <IP_KONCOWEGO_HOSTA> # UDP
tracepath <IP_KONCOWEGO_HOSTA>
mtr <IP_KONCOWEGO_HOSTA>

ip addr
ip route
ip neigh
sudo iptables -L -n -v -x --line-numbers
sudo iptables -t nat -L -n -v -x --line-numbers
wireshark
sudo zypper install <pakiet>
```

Najpierw pinguj bezpośredniego sąsiada. Jeśli to nie działa, nie debuguj jeszcze routingu ani NAT: problem jest w kablu, porcie, VLANie, masce, IP albo ARP. Traceroute działa przez TTL; brak odpowiedzi z jednego hopa może oznaczać filtr ICMP, a nie brak trasy.

więc przechodzi przez FORWARD, nie INPUT.

- **Lokalny test z routera używa OUTPUT, nie PREROUTING.** Typowe zadanie testuje z innego PC, więc PREROUTING wystarcza.

Włączenie routera NAT

```
sudo sysctl -w net.ipv4.ip_forward=1
# sprawdź trasy:
ip route
# sprawdź conntrack/reguły:
sudo iptables -t nat -L -n -v -x --line-numbers
sudo iptables -L FORWARD -n -v -x --line-numbers
```

SNAT i MASQUERADE - gotowce

Internet dla sieci prywatnej przez em1

```
# PC-router: p4p1 = LAN 192.168.1.1/24, em1 = internet/DHCP
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -F
sudo iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o em1 -j MASQUERADE
```

Na hostach LAN: `sudo ip route add default via 192.168.1.1`. DNS: skopiuj/ustaw `/etc/resolv.conf`, jeśli ping po IP działa, a po nazwie nie.

Klasyczny SNAT na stały adres routera

```
sudo iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o em1 -j SNAT --to-source 150.254.32.66
```

Użyj SNAT, gdy adres zewnętrzny jest znany i stały. Użyj MASQUERADE, gdy interfejs ma DHCP albo może zmienić adres.

SNAT między dwiema prywatnymi sieciami

Przykład: PC2 łączy PC1 192.168.1.0/24 i PC3 192.168.111.0/24. PC3 ma default do PC2, PC1 nie ma trasy do 192.168.111.0/24. Żeby PC3 mógł łączyć się z PC1 bez dodawania trasy na PC1:

```
# na PC2, interfejs do PC1 ma adres 192.168.1.2
sudo iptables -t nat -A POSTROUTING \
-s 192.168.111.0/24 -d 192.168.1.0/24 \
-j SNAT --to-source 192.168.1.2
```

PC1 widzi ruch tak, jakby pochodził z PC2 192.168.1.2, więc odpowiada do PC2, a conntrack odłumacza odpowiedź do PC3.

DNAT - przekierowanie portów

Prosty port forward do hosta za NATem

Założenie: PC1 łączy się z adresem PC2 od strony PC1, a usługa działa na PC3 192.168.111.1:74. PC2 ma forwarding.

```
# na PC2
sudo iptables -t nat -A PREROUTING -i p4p1 -p tcp --dport 74 \
-j DNAT --to-destination 192.168.111.1:74
```

Test:

```
# PC3
ncat -l 192.168.111.1 74
# PC1 łączy się z adresem PC2 w sieci PC1-PC2
ncat 192.168.1.2 74
```

PC1 nie musi znać PC3. PC3 musi umieć odpowiedzieć przez PC2, np. default via 192.168.111.2.

ARP i DHCP

```
ip neigh show
sudo ip neigh flush dev p4p1
sudo ip neigh add 192.168.1.10 lladdr aa:bb:cc:dd:ee:ff dev p4p1
sudo ip neigh del 192.168.1.10 dev p4p1
sudo arping -I p4p1 192.168.1.10
sudo ip link set arp off dev p4p1
sudo ip link set arp on dev p4p1
arp
sudo arp -i p4p1 -s 192.168.1.10 aa:bb:cc:dd:ee:ff
sudo arp -i p4p1 -d 192.168.1.10
cat /proc/sys/net/ipv4/neigh/p4p1/gc_stale_time

# DHCP, jeśli trzeba odnowić em1/p4p1
sudo dhclient -r -v em1
sudo dhclient -v em1
journalctl -f
cat /etc/resolv.conf
cat /var/lib/dhclient/dhclient.leases # czasem /var/lib/dhcp/...
```

ARP działa tylko w lokalnej domenie L2/VLAN. Jeśli pingujesz host z innej sieci, ARP powinien być do bramy, nie do hosta docelowego.

Cisco routery: tryby, adresy, routing

Powłoka i diagnostyka

```
Router> enable
Router# configure terminal
Router(config)# hostname R1
R1(config)# end # albo Ctrl-Z
R1# show running-config
R1# show interfaces
R1# show interface GigabitEthernet 0/1
R1# show ip interface brief # nie pokazuje secondary!
R1# show ip route
R1# show cdp neighbors
R1# show controllers Serial 0/1/0
R1# ping 192.168.1.2
R1# traceroute 192.168.3.2 numeric
```

Skróty: `?`, `Tab`, do `show ...` w trybie konfiguracji, `no <komenda>` odwraca działanie, `no shutdown` podnosi interfejs.

Adresy na interfejsach

```
R1# conf t
R1(config)# interface GigabitEthernet 4
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# interface GigabitEthernet 5
R1(config-if)# ip address 10.0.0.1 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# end
```

Routery z labu: Gi4/Gi5 są zwykłymi portami L3. Gi0-Gi3 mogą być za wewnętrznym switchem; wtedy adres nadaje się na interface Vlan1, a kabel można wpiąć w dowolny Gi0-Gi3.

Jeśli pojawi się łącze szeregowo, adresujesz interface Serial ...; clock rate 128000 ustawiasz tylko na końcówce DCE, co sprawdzisz przez `show controllers Serial ...`.

```
R1# conf t
R1(config)# interface Serial 0/1/0
R1(config-if)# ip address 10.0.0.1 255.255.0.0
R1(config-if)# clock rate 128000
R1(config-if)# no shutdown
```

Kilka adresów bez VLANów: secondary

```
R1# conf t
R1(config)# interface FastEthernet 0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# ip address 172.16.13.13 255.255.128.0 secondary
R1(config-if)# ip address 10.10.10.10 255.255.240.0 secondary
R1(config-if)# no shutdown
R1(config-if)# do show ip interface FastEthernet 0/0
```

Uwaga: show ip interface brief pokaże tylko adres główny, nie secondary.

Routing statyczny Cisco

```
R1# show ip route
R1# conf t
R1(config)# ip route 172.16.0.0 255.255.0.0 192.168.1.111
R1(config)# ip route 0.0.0.0 0.0.0.0 192.168.1.1
R1(config)# no ip route 172.16.0.0 255.255.0.0 192.168.1.111
```

Pakiet zwrotny też musi mieć trasę. Jeśli ping działa tylko w jedną stronę w Wiresharku, prawie zawsze brakuje routingu powrotnego albo filtr/NAT go zmienia.

OSPF Cisco - gdy pojawi się routing dynamiczny

OSPF zastępuje ręczne wpisywanie tras statycznych na każdym routerze. W zadaniu zwykle chodzi o jedno area 0: każdy router ma ogłaszać swoje bezpośrednio podłączone sieci, a sąsiedzi mają nauczyć się tras automatycznie.

Procedura konfiguracji

- 1. Skonfiguruj IP na interfejsach i sprawdź ping do bezpośrednich sąsiadów.
- 2. Na każdym routerze włącz router ospf 1. Numer procesu jest lokalny; dla porządku wszędzie użyj 1.
- 3. Dodaj network <SIEC> <WILDCARD> area 0 dla sieci, które mają wejść do OSPF. Ta koda włącza OSPF na pasujących interfejsach i rozgłasza te sieci.
- 4. Sprawdź show ip ospf neighbor. Bez sąsiadów nie będzie tras OSPF w show ip route.

```
R1# show ip interface brief
R1# ping <IP_BEZPOSREDNIEGO_SASIADA>
R1# conf t
R1(config)# router ospf 1
R1(config-router)# network 10.0.12.0 0.0.0.255 area 0
R1(config-router)# network 10.0.13.0 0.0.0.255 area 0
R1(config-router)# end
R1# show ip ospf neighbor
R1# show ip protocols
R1# show ip ospf
R1# show ip ospf interface
R1# show ip ospf database
R1# show ip ospf database network
R1# show ip ospf database router
R1# show ip route
```

Wildcard mask to odwrotność maski: /24 = 0.0.0.255, /30 = 0.0.0.3, /16 = 0.0.255.255. Jeśli chcesz objąć wszystkie sieci 10.0.*.*, możesz użyć network 10.0.0.0 0.0.255.255 area 0, ale precyzyjne wpisy są łatwiejsze do debugowania.

Bezpieczny szkielet skryptu

```
#!/bin/bash
set -e

# 1. Najpierw otwórz polityki, potem czyść reguły.
sudo iptables -F INPUT ACCEPT
sudo iptables -F FORWARD ACCEPT
sudo iptables -F OUTPUT ACCEPT
sudo iptables -F
sudo iptables -t nat -F
sudo iptables -t mangle -F
sudo iptables -X
sudo iptables -Z

# 2. Polityki docelowe.
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD ACCEPT
sudo iptables -P OUTPUT ACCEPT

# 3. Reguły od najbardziej szczegółowych do ogólnych.
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# wpuszcza odpowiedzi na połączenia rozpoczęte przez ten komputer
sudo iptables -A INPUT -i lo -j ACCEPT
# zostawia loopback, potrzebny lokalnym programom
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
# pozwala innym hostom pingować ten komputer
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
# pozwala na SSH do tego komputera
```

Uwaga: Jeśli pracujesz lokalnie w labie, nie ma ryzyka utraty SSH do własnego PC, ale kolejność nadal ma znaczenie.

NAT: zasady i niuanse

Co zmienia SNAT/MASQUERADE/DNAT

Mechanizm	Gdzie	Sens
SNAT	nat POSTROUTING	Zmienia adres/port źródłowy pakietu wychodzącego. Używany, gdy znasz adres zewnętrzny routera.
MASQUERADE	nat POSTROUTING	SNAT automatycznie na adres interfejsu -o. Dobre dla DHCP/zmiennego IP.
DNAT	nat PREROUTING	Zmienia adres/port docelowy przed routin-giem. Używany do przekierowania portu do hosta za NATem.

Najważniejsze niuanse NAT

- **NAT nie zastępuje routingu.** Router musi mieć trasy, hosty muszą mieć bramę zwrotną, forwarding musi być włączony.
- **SNAT i DNAT są niezależne.** SNAT: zmiana źródła w żądaniu, automatyczna zmiana celu w odpowiedzi. DNAT: zmiana celu w żądaniu, automatyczna zmiana źródła w odpowiedzi.
- **NAT działa stanowo.** Conntrack pamięta mapowania, także translację portów, gdy dwa hosty prywatne użyją tego samego portu źródłowego.
- **Zawężaj reguły.** Prawie zawsze dodaj -s, -d, -i, -o, -p, --dport. Zbyt ogólny SNAT na wszystkich interfejsach utrudnia debugowanie i może psuć komunikację lokalną.
- **DNAT wymaga drogi powrotnej.** Host docelowy za NATem powinien mieć default przez router NAT albo trzeba dodać SNAT, żeby odpowiedź wróciła przez NAT.
- **FORWARD może blokować DNAT.** Po DNAT pakiet jest routowany do hosta prywatnego,

```
# dodawanie/usuwanie/czyszczenie
sudo iptables -A INPUT ...      # append na koniec
sudo iptables -I INPUT 1 ...     # insert na pozycje 1
sudo iptables -D INPUT 3        # usun 3. regule
sudo iptables -D INPUT ...      # usun regule po tresci
sudo iptables -F INPUT          # flush lancucha
sudo iptables -F                # flush filter
sudo iptables -t nat -F         # flush nat
sudo iptables -X                # usun wlasne lancuchy
sudo iptables -Z                # wyzeruj liczniki
```

iptables: match-e, conntrack, connlimit

Najczęstsze dopasowania

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

-s 192.168.1.0/24      # source
-d 10.0.0.5            # destination
-i p4pl                # interfejs wejscowy
-o em1                 # interfejs wyjscowy
-p tcp                # tcp/udp/icmp
-p tcp --dport 22      # port docelowy
-p tcp --sport 12345   # port zrodlowy
-p udp --dport 53
-p icmp --icmp-type echo-request
! -s 192.168.1.55      # negacja
-m mac --mac-source aa:bb:cc:dd:ee:ff
-m owner --gid-owner 1006
-m comment --comment "opis reguly"

sudo iptables -I INPUT 1 -p tcp -s 192.168.1.55 --dport 22 -j REJECT
sudo iptables -A OUTPUT -p tcp -d 150.254.30.30 -j DROP
sudo iptables -A INPUT -p tcp --syn -j DROP
sudo iptables -A INPUT -p tcp --tcp-flags SYN,RST,ACK,FIN SYN -j DROP
sudo iptables -A INPUT -m mac ! --mac-source aa:bb:cc:dd:ee:ff -j DROP
sudo iptables -A OUTPUT -m owner --gid-owner 1006 ! -d 192.168.1.0/24 -j DROP
```

Pierwsza reguła z `conntrack` prawie zawsze powinna być wysoko: wpuszcza odpowiedzi na połączenia, które sam zainicjowałeś, np. odpowiedź SSH, DNS albo ping. Reguła z nazwą DNS w `-d` jest rozwijana tylko przy dodawaniu reguły; do testu zwykle lepiej wpisać IP. `owner` działa dla pakietów lokalnie generowanych, zwykle w `OUTPUT`, a nie dla routowanych przez `FORWARD`.

Conntrack i connlimit krótko

```
# router z FORWARD DROP: LAN wychodzi, odpowiedzi wracaja
sudo iptables -P FORWARD DROP
sudo iptables -A FORWARD -i p4pl -o em1 -s 192.168.1.0/24 \
-m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A FORWARD -i em1 -o p4pl -d 192.168.1.0/24 \
-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# maksymalnie 2 polaczenia SSH z jednego IP klienta
sudo iptables -A INPUT -p tcp --dport 22 \
-m connlimit --connlimit-above 2 -j DROP

# maksymalnie 2 polaczenia SSH lacznice, ze wszystkich IP
sudo iptables -A INPUT -p tcp --dport 22 \
-m connlimit --connlimit-above 2 --connlimit-mask 0 -j DROP
```

`NEW` to początek połączenia, `ESTABLISHED` to pakiet należący do znanego połączenia, `RELATED` to ruch powiązany, np. część błędów ICMP. `connlimit` musi stać przed ogólnym `ACCEPT` dla SSH, inaczej limit nigdy się nie uruchomi.

Default i koszty

```
# tylko na routerze, ktory ma wyjscie dalej, np. do Internetu:
R1# conf t
R1(config)# ip route 0.0.0.0 0.0.0.0 <NEXT_HOP>
R1(config)# router ospf 1
R1(config-router)# default-information originate

# opcjonalnie, gdy zadanie zkae czmieni metryki:
R1(config-router)# auto-cost reference-bandwidth 1000
R1(config-router)# exit
R1(config)# interface GigabitEthernet 0/1
R1(config-if)# bandwidth 25000      # 25 Mb/s; IOS zwykle podaje kb/s
R1(config-if)# ip ospf cost 17      # reczny koszt OSPF, nadpisuje bandwidth
```

`show ip route` oznacza trasy OSPF literą O. OSPF ma dystans administracyjny 110, więc trasa statyczna o tym samym celu zwykle wygra z OSPF.

VLANy i router-on-a-stick

Zasady

VLAN = osobna domena rozgłoszeniowa w warstwie 2. Hosty w różnych VLANach nie komunikują się bez routera, nawet jeśli wpiszesz im adresy z tej samej sieci IP. Połączenie przenoszące wiele VLANów to **trunk**; ramki dostają tag 802.1Q. Port do zwykłego PC jest **access**.

Najpierw rozpoznaj porty

Nie ucz się numerów portów na pamięć. Najpierw sprawdź, jak urządzenie nazywa porty, a potem podstaw właściwe nazwy w miejsce `<PORT_PC>` i `<PORT_TRUNK>`.

```
S1# show ip interface brief
S1# show interface status
S1# show interfaces status
S1# show vlan
S1# show vlan brief
S1# show vlan-switch
S1# show interfaces trunk
```

Jeśli `show vlan` jest niejednoznaczne, wpisz `show vlan ?`. Na części starszych Cisco/EtherSwitch właściwa komenda to `show vlan-switch`; na części nowszych wystarcza `show vlan` albo `show vlan brief`. W wyjściu nowych/białych switchy porty `tagged` zwykle oznaczają trunk, a `untagged` port `access`.

Tworzenie VLANów: normalny IOS

```
S1# conf t
S1(config)# vlan <VLAN_ID>
S1(config-vlan)# name <NAZWA>
S1(config-vlan)# exit
S1(config)# end
```

`vlan <VLAN_ID>` tworzy albo wybiera VLAN. `name` nadaje tylko etykietę dla człowieka; separację robi numer VLAN. Domyślnie VLAN jest aktywny.

Tworzenie VLANów: starszy vlan database

Na niektórych starych/czarnych switchach `vlan database` działa z trybu uprzywilejowanego `S1\#`, a nie z `conf t`. W tym trybie `end` może być błędem; wychodzisz przez `exit`, co zapisuje zmiany.

```
S1> enable
S1# vlan database
S1(vlan)# vlan <VLAN_ID> name <NAZWA>
S1(vlan)# exit
```

Po utworzeniu VLANów wracasz do zwykłego `conf t` i dopiero tam przypisujesz porty.

Port access dla zwykłego hosta

```
S1# conf t
S1(config)# interface <PORT_PC>
S1(config-if)# switchport mode access
S1(config-if)# switchport access vlan <VLAN_ID>
S1(config-if)# no shutdown
S1(config-if)# end
```

`interface <PORT_PC>` wybiera port z listy `show ...switchport mode access` wymusza tryb jednego, nietagowanego VLANu. `switchport access vlan <VLAN_ID>` przypisuje ten port do VLANu hosta; samo nie robi trunku.

Port trunk między switchami albo do routera

```
S1# conf t
S1(config)# interface <PORT_TRUNK>
S1(config-if)# switchport trunk encapsulation dot1q
S1(config-if)# switchport mode trunk
S1(config-if)# no shutdown
S1(config-if)# end
S1# show interfaces trunk
```

`switchport mode trunk` przenosi wiele VLANów przez jeden kabel. `switchport trunk encapsulation dot1q` jest potrzebne tylko na starszych urządzeniach, które pozwalają wybrać enkapsulację; jeśli komenda jest nieznana, zwykle `dot1q` jest jedyną opcją i przechodzisz dalej.

Router między VLANami

Port router-switch też musi być trunk. Na routerze nie nadajesz IP na fizycznym interfejsie, tylko na podinterfejsach.

```
R1# conf t
R1(config)# interface <PORT_DO_SWITCHA>
R1(config-if)# no ip address
R1(config-if)# no shutdown
R1(config-if)# interface <PORT_DO_SWITCHA>.<VLAN_A>
R1(config-subif)# encapsulation dot1q <VLAN_A>
R1(config-subif)# ip address <BRAMA_A> <MASKA_A>
R1(config-subif)# interface <PORT_DO_SWITCHA>.<VLAN_B>
R1(config-subif)# encapsulation dot1q <VLAN_B>
R1(config-subif)# ip address <BRAMA_B> <MASKA_B>
R1(config-subif)# end
R1# show ip interface brief
```

`encapsulation dot1q <VLAN>` mówi, jaki tag 802.1Q obsługuje dany podinterfejs. Adres IP podinterfejsu jest bramą domyślną dla hostów w tym VLANie. Jeśli dwa switche przenoszą VLANy, wszystkie połączenia między nimi muszą być trunkami.

```
# Bez ACL router Cisco routuje miedzy VLANami domyslnie.
# Jesli ACL filtruje ruch, dopusc oba kierunki przed deny:
R1# conf t
R1(config)# ip access-list extended VLANY
R1(config-ext-nacl)# permit ip <SIEC_A> <WILD_A> <SIEC_B> <WILD_B>
R1(config-ext-nacl)# permit ip <SIEC_B> <WILD_B> <SIEC_A> <WILD_A>
R1(config-ext-nacl)# exit
R1(config)# interface <PORT_DO_SWITCHA>.<VLAN_A>
```

```
R1(config-subif)# ip access-group VLANY in
R1(config-subif)# interface <PORT_DO_SWITCHA>.<VLAN_B>
R1(config-subif)# ip access-group VLANY in
```

Uwaga: ACL Cisco jest bezstanowa i ma ukrytą `ip any any` na końcu. Jeśli chcesz zwykły routing między VLANami, nie zakładaj ACL. Wildcard: `/24 = 0.0.0.255`.

iptables: model mentalny

Netfilter, tabele i łańcuchy

Tabela	Do czego
filter	firewall: przepuszcza/blokuje pakiety. Łańcuchy: INPUT, FORWARD, OUTPUT.
nat	translacja adresów/portów. Łańcuchy: PREROUTING, OUTPUT, POSTROUTING.
mangle	modyfikacje pakietu, np. TTL. Łańcuchy: PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING.

Przepływ pakietu przez Linuxa

- Do samego Linuxa: PREROUTING -> routing -> INPUT.
- Wygenerowany lokalnie: OUTPUT -> routing -> POSTROUTING.
- Przekazywany przez router: PREROUTING -> routing -> FORWARD -> POSTROUTING.

Z tego wynika: INPUT nie filtruje ruchu przechodzącego przez router; do tego jest FORWARD. PREROUTING nie zna jeszcze interfejsu wyjściowego -o. POSTROUTING nie używa -i. DNAT zwykle robi się w PREROUTING, SNAT/MASQUERADE w POSTROUTING.

Najważniejsze zasady

- Reguły są sprawdzane od góry. Pierwsza decydująca reguła kończy przetwarzanie danego łańcucha.
- DROP ignoruje pakiet; klient czeka/timeout. REJECT odrzuca aktywnie, zwykle szybciej widać błąd; w Wiresharku pojawi się odpowiedź ICMP albo TCP RST zależnie od protokołu/opcji.
- Polityka -P działa, gdy żadna reguła nie pasuje. Na zaliczeniu blokowanie całego ruchu wejściowego rób polityką: `sudo iptables -P INPUT DROP`.
- NAT korzysta z conntracka: odpowiedzi są automatycznie odtłumaczane. Reguła NAT zwykle trafia tylko pierwszy pakiet połączenia; potem działa stan połączenia.
- Dla routera z firewallem pamiętaj o FORWARD; przy polityce FORWARD DROP musisz dopuścić ruch routowany i powrotny.

iptables: składnia, reset, liczniki

Składnia i operacje

```
sudo iptables [-t tabela] OPCJA LANCUCH [match...] -j CEL

# listowanie
sudo iptables -L
sudo iptables -t nat -L
sudo iptables -t mangle -L
sudo iptables -t filter -L -n -v -x --line-numbers
sudo iptables -S
sudo iptables -t nat -S
sudo iptables-save
sudo iptables -I INPUT -p tcp --help # pomoc dla opcji TCP

# polityki
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD ACCEPT
sudo iptables -P OUTPUT ACCEPT
```